

Swift
Master Test Plan and Test Report
Version 3.0

Revision History

Date	Version	Description	Author
20 Sep 2026	1.0	Initial full-system plan, execution evidence, dataset evaluation and error analysis	Swift project team / Codex
20 Sep 2026	2.0	Severe defects remediated; regression, accessibility, load and security evidence refreshed	Swift project team / Codex
20 Sep 2026	3.0	Results re-executed and evidence retained; coverage metrics corrected; requirement traceability added	Swift project team / Claude

Table of Contents

Document Control	3
Executive Test Summary	3
Release recommendation	4
1. Evaluation Mission and Test Motivation	4
2. Target Test Items	4
3. Test Approach	5
3.1 Testing Techniques and Types	5
3.1.1 Data and Database Integrity Testing	5
3.1.2 Function Testing	6
3.1.3 User Interface Testing	8
3.1.4 Performance Profiling	9
3.1.5 Load Testing	10
3.1.6 Security and Access Control Testing	11
3.1.7 Failover and Recovery Testing	13
3.1.8 Configuration Testing	14
4. Deliverables	15
4.1 Test Evaluation Summaries	15
4.2 Reporting on Test Coverage	15
4.3 Requirement Traceability Matrix	16
5. Risks, Dependencies, Assumptions, and Constraints	18
5.1 Defect register	18
Defects closed and verified	21
5.2 Risks, mitigation and contingency	22
5.3 Dependencies	22
5.4 Assumptions and constraints	23
5.5 Acceptance decision	23
5.6 Ordered remediation and retest plan	23
6. References	24

Document Control

Field	Value
System under test	Swift — multilingual, multimodal banking-support ticket triage
Build under test	e6ef318
Execution date	20 September 2026
Test environment	Developer workstation, Python 3.13, Node 22 (backend and frontend suites, static analysis, offline diagnostics); containerised Linux stack, Python 3.12, with PostgreSQL 16/pgvector, Redis and a local generation provider (API, browser, load and dynamic-scan evidence)
Overall verdict	CONDITIONAL PASS — supportable for controlled, human-supervised demonstration; not production-ready

Executive Test Summary

Area	Result
Backend suite	195 passed, 2 failed, 3 skipped (199 collected); no expected-failure markers remain
Backend coverage	71.72% statement, 43.41% branch
Backend static quality	Ruff clean; mypy strict clean across 38 source files
Frontend suite	45 of 45 passed across 18 files
Frontend coverage	21.82% line, 19.59% statement, 19.14% branch, 15.25% function
Frontend gates	ESLint, TypeScript no-emit and production build all pass
JavaScript dependency audit	0 vulnerabilities at every severity
API business flow	10 requests, 23 assertions, 0 failures
Load, read-only endpoints	2,000 samples, 0 errors, mean 17.0 ms, p95 39 ms
Browser and GUI	Cypress 3 of 3; Selenium desktop and 390×844 mobile
Accessibility	axe 0 violations, 31 rules passed; Lighthouse accessibility 100
Web performance	Lighthouse performance 95; FCP 2.2 s, LCP 2.5 s, CLS 0
Prompt-injection guardrail	18 of 18 detected; ticket-context scanning gap closed
Multilingual safety parity	840 of 841 paired tickets escalate in every language track
Rate limiting	8 rate-limit and lockout cases pass
Dynamic scan, API	0 medium or high findings; 4 informational
Dynamic scan, frontend	3 medium-risk, high-confidence findings
Dataset integrity	10 of 10 files pass schema, identifier and cross-language label alignment
Classifier metrics	Intent macro-F1 0.8834; sentiment Negative-F1 0.7138; priority macro-F1 0.8734
Language identification	English, Sinhala and Tamil 100%; Singlish 91.56%; Tamilish 25.63%
Python SAST and dependency audit	Not executed in this cycle
RAG retrieval and answer quality	Not executed in this cycle

Release recommendation

The severe defects recorded in the previous cycle are closed, and the most important closures were confirmed by re-executing the diagnostics that originally measured them. The prompt-injection corpus is now fully detected where five sixths of it previously passed through; the unscanned ticket-context path is closed; multilingual safety parity moved from under 4% to 840 of 841 paired tickets; rate limiting is implemented and tested; and the cross-language dataset label defect is fixed.

A controlled, human-supervised demonstration is supportable, provided external financial actions remain disabled and every generated draft is approved by a person before it reaches a customer.

Three findings prevent a stronger recommendation:

1. The load evidence covers five read-only endpoints. Ticket creation and assistance generation — measured at 2.1 s and 26.6 s respectively for a single user — have not been placed under concurrent load, so no capacity claim covers the paths that dominate cost and latency.
2. RAG answer quality, citation correctness and generation latency were not measured in this cycle. Only the safety behaviour of the retrieval path is established.
3. Three medium-risk web-hardening findings are open against the frontend, and one ticket in the paired multilingual corpus still escalates in three language tracks but not in the other two.

1. Evaluation Mission and Test Motivation

Swift accepts banking-support tickets in English, Sinhala, Singlish, Tamil and Tamilish, through both text and image paths with OCR. It classifies intent, sentiment and priority, enforces customer, agent and administrator separation, and supports human agents with retrieval-grounded draft responses. It handles security-sensitive banking language and customer-uploaded material.

Because the system is multilingual and advisory, a defect can cause harm that the user interface will not reveal. A guardrail that fires on an English ticket but not on its Tamil translation produces no error and no failing test; it produces two customers treated differently. Detecting that class of fault, rather than conventional functional failure, is the principal motivation for this test effort.

The mission for this iteration is to:

1. Confirm that the severe defects recorded in the previous cycle are closed in code, by re-running the diagnostics that originally measured them.
2. Re-execute every gate that can run without paid third-party APIs or dedicated hardware, and retain the output as evidence.
3. Establish requirement-level traceability from use cases to tests, results and evidence.
4. Quantify residual quality risk and state which parts of the system remain unmeasured.

The fault model of interest is language-dependent divergence. The five language tracks are translations of one ticket set keyed by identifier, so any rule that is correct must fire on all five renderings or on none. A per-language gap in safety routing, classification or detection is treated as a defect even when no test fails.

2. Target Test Items

Test item	Technology	Criticality	Covered in this cycle
Customer, agent and administrator web interface	React 19, Vite 7, Nginx	High	Component, browser, responsive, accessibility, build
Authentication, tickets, attachments, dashboards, RAG API	FastAPI, SQLAlchemy	Critical	Unit, API, contract, integration, security
Persistent data and migrations	PostgreSQL 16, pgvector, Alembic	Critical	Constraints, transactions, injection, migration graph
Background worker boundary	Dramatiq, Redis	High	Configuration resolution and container health

Test item	Technology	Criticality	Covered in this cycle
Multimodal OCR and masking path	Tesseract, masking endpoint	Critical	Unit and API level
Container deployment	Docker Compose	High	Configuration validation, build and startup
Intent, sentiment and priority models	LaBSE, TF-IDF with SVM	Critical	Metrics recomputed from saved prediction artifacts
Multilingual datasets and frozen split	Five language and script tracks	Critical	Full non-mutating audit
RAG retrieval, guardrails and citations	BGE-M3, pgvector, full-text search, reranker, local generation	Critical	Injection and safety routing measured; citation structure and answer quality not measured
External generation and OCR providers	Third-party APIs	High	Not called in this cycle
Bank core, payment execution, notifications	Not implemented	N/A	Outside the system boundary
Native desktop and mobile applications	Not present; responsive web only	N/A	Not applicable

3. Test Approach

The approach combines white-box automated tests, black-box HTTP and browser checks, static analysis and type checking, passive dynamic scanning, concurrent load, container validation, deterministic non-mutating artifact audits, and manual review.

Results are reported per technique with an explicit success criterion and an explicit basis. Where a technique was not executed in this cycle, it is reported as not executed rather than omitted, so that the untested surface is visible in the same tables as the tested surface.

3.1 Testing Techniques and Types

3.1.1 Data and Database Integrity Testing

Technique Objective	Exercise database access methods, migration history and dataset integrity independently of the user interface, so that data corruption and cross-language label divergence are observable without going through the application.
Technique	Run the database suite against isolated fixtures; walk the migration revision graph and assert a single reachable head with working downgrades; assert ORM and migration agreement, uniqueness, cascades, audit retention, rollback and atomic creation; drive five SQL-injection payload families through customer and administrator search; run the non-mutating dataset audit and the cleaning audit over all ten labelled files.
Oracles	Self-verifying. Database assertions pass or fail in the test runner. The dataset audit emits a machine-readable pass or fail per control, with an overall structural pass as the summary oracle; the cleaning audit emits counted findings.
Required Tools	Test runner with a database marker; SQLAlchemy;

	Alembic; the dataset validation script; the cleaning audit script; Compose configuration validation.
Success Criteria	One reachable migration head; expected constraints and transaction behaviour; every dataset file shares the required schema; identifiers align across all five language tracks; labels agree by identifier across tracks.
Special Considerations	The dataset audit must not mutate source files. Database tests run on isolated fixtures, so pgvector behaviour and PostgreSQL-specific SQL are covered by the container evidence rather than by the suite.

Results. The database suite passes, with one failure attributable to a cross-platform defect recorded in §3.1.8.

Dataset audit, overall structural pass:

Control	Result
Schema id, text_en, text, category, sentiment, priority	Pass, 10 of 10 files
Row counts	Pass: 9,998 train and 3,079 test per language
Unique identifiers within each file	Pass
77 intent categories per file	Pass
Identifier alignment across tracks	Pass
Source-text alignment	Pass
Cross-language label alignment	Pass — the previously recorded Tamilish mismatch is closed
Frozen split	Pass: 8,500 / 1,498 / 3,079, manifest e7b5934392cd
Experiment run records	Pass: 1,168 of 1,168 parse and carry the frozen split manifest

Cleaning audit: zero label mismatches against English, zero empty fields, zero untranslated rows. Four findings remain open for human review rather than silent deduplication:

- Train and test leakage: 6 test rows whose English source text also appears in train.
- Conflicting-label collisions: 1 group, 4 member rows, in Tamilish train and test.
- Exact duplicates, same text and same labels: Tamil 120 rows, Tamilish 98 rows, 218 in total.
- Class imbalance: sentiment is 95.4% Neutral to 4.6% Negative, a ratio of 20.6 to 1; priority is 53.1% Low, 36.9% Medium, 9.9% High. This is why Negative-F1 governs sentiment in §3.1.4.

3.1.2 Function Testing

Technique Objective	Exercise business rules, authentication and authorisation, attachment handling, policy routing, OCR-assisted intent behaviour and error responses, observing behaviour through the API surface rather than through internals.
Technique	Execute the backend suite covering unit, API, contract, integration, security, observability and inference; execute the frontend component suite; drive a ten-step business flow against a running deployment, covering health, readiness, contract, anonymous rejection, invalid registration,

	registration, authenticated profile, ticket creation with classification, ownership-scoped retrieval, and assistance generation.
Oracles	Self-verifying. Test runner results; business-flow assertion counts with explicit status-code and body assertions at each step.
Required Tools	Python test runner with coverage and property-based testing; Vitest with React Testing Library; Postman collection executed through Newman.
Success Criteria	All key use-case scenarios and features execute; valid data produces the expected result; invalid data produces the appropriate error; each business rule is applied; no expected-failure markers remain outstanding.
Special Considerations	Two golden retrieval tests require a live database with an ingested knowledge base and a generation provider, and skip without them. The contract fuzzing test requires an optional dependency that is not part of the default environment.

Results — backend:

Outcome	Count
Collected	199
Passed	195
Failed	2
Skipped	3
Expected failures	0

No expected-failure markers remain in the suite. The previous cycle carried 23 strict expected failures — one contract drift, 16 prompt-injection cases and six missing rate-limit controls — and all are now implemented and passing rather than deferred.

The three skips are declared by the tests themselves: one optional contract-fuzzing dependency, and two golden retrieval tests requiring a live database and generation provider. The two failures are cross-platform defects recorded in §3.1.8.

Results — frontend: 45 passed of 45 across 18 test files.

Results — API business flow: 10 requests, 23 assertions, 0 failures.

Step	Status	Time
Health endpoint	200	15 ms
Database readiness	200	4 ms
OpenAPI document	200	48 ms
Protected endpoint rejects anonymous caller	401	4 ms
Invalid registration rejected	422	3 ms
Register customer	201	44 ms
Authenticated profile	200	5 ms
Create support ticket with classification	201	2,065 ms

Step	Status	Time
Retrieve owned ticket	200	11 ms
Assistance generation	200	26,573 ms

Every assertion passes, but the assistance step takes 26.6 seconds synchronously. This is a usability defect and is recorded as RAG-R1.

3.1.3 User Interface Testing

Technique Objective	Verify that navigation, form validation and responsive behaviour work through the rendered interface, and that the production build satisfies automated accessibility rules at desktop and mobile widths.
Technique	Drive the production frontend at 1280×800 and 390×844, covering login validation, responsive and theme switching, and registration role switching; run a headless browser smoke test at both widths asserting title, heading and form controls, and capture a mobile screenshot; run automated accessibility rules against the production login route; run a page-quality audit in a headless browser; review the captured screenshot manually for clipping and overflow.
Oracles	Self-verifying for the automated layers: browser test results, accessibility violation array, audit category scores. Manual screenshot review is a human oracle and is reported as such.
Required Tools	Cypress 14.5.4; Selenium with headless Chrome; axe-core 4.13.0; Lighthouse 12.8.2.
Success Criteria	All browser assertions pass at both widths; zero accessibility violations; accessibility score at or above 90; no clipping or horizontal overflow on manual review.
Special Considerations	Automated rules cover only part of WCAG. No screen-reader pass, keyboard-only traversal, 200–400% zoom and reflow check, or Sinhala and Tamil pronunciation review was performed.

Results.

Check	Result
Cypress	3 of 3 passed, 1.65 s, 0 failures
Selenium, desktop and 390×844 mobile	Passed; screenshot retained
axe-core on the login route	0 violations, 31 rules passed, 58 inapplicable, 1 item requiring manual review
Lighthouse accessibility	100

The previously recorded accessibility defects — a missing level-one heading and undersized touch targets — are closed. The one remaining item is an accessibility rule that cannot be decided automatically and has not yet been manually adjudicated; it is recorded as A11Y-02 rather than counted as a pass.

3.1.4 Performance Profiling

Technique Objective	Measure single-user page performance and model-quality characteristics, to identify where latency and error are concentrated before capacity is considered.
Technique	Profile the production login route in a headless browser at mobile form factor; recompute classifier metrics from saved row-level prediction files rather than from narrative reports; recompute per-language error rates; re-run the offline language-identification diagnostic.
Oracles	Self-verifying. Audit category scores and Core Web Vitals; metric recomputation from prediction files with row counts, split manifest and label version recorded alongside the results.
Required Tools	Lighthouse 12.8.2; the dataset validation script; the offline language diagnostic.
Success Criteria	Performance score at or above 90 with cumulative layout shift near zero; recomputed metrics reproduce the published headline figures at the stated label version and split.
Special Considerations	Page measurements are local synthetic results on one machine, not field data. Sentiment and priority metrics measure agreement with the active labelling prompt version, not human ground truth.

Results — page performance, mobile form factor:

Metric	Result
Performance	95
Accessibility	100
Best practices	96
SEO	100
First Contentful Paint	2.2 s
Largest Contentful Paint	2.5 s
Total Blocking Time	10 ms
Cumulative Layout Shift	0

This closes the previously recorded finding of a performance score of 76 with a 4.0 s First Contentful Paint.

Results — classifiers, recomputed over 15,395 pooled test rows:

Task and model	Primary metric	Accuracy
Intent, LaBSE	macro-F1 0.883424	0.882949
Intent, TF-IDF with SVM	macro-F1 0.830820	0.830464
Sentiment, LaBSE	Negative-F1 0.713819	0.965833
Sentiment, TF-IDF with SVM	Negative-F1 0.665291	0.957843
Priority, TF-IDF with SVM	macro-F1 0.873358	0.887886

Negative-F1 governs sentiment because the class distribution is 95.4% Neutral: a classifier predicting only Neutral reaches roughly 95.6% accuracy while detecting no negative tickets. The published results now headline the current label version and mark earlier-version rows as not comparable, closing the previously recorded reporting inconsistency.

Results — per-language intent error rate:

Language	LaBSE	TF-IDF with SVM
English	5.88%	8.22%
Sinhala	6.79%	13.19%
Tamil	6.72%	14.23%
Singlish	9.65%	12.02%
Tamilish	29.49%	37.12%

Tamilish is the dominant model weakness, 23.61 percentage points worse than English. Non-standard romanisation removes script cues, and the Tamilish track has not had a human correction pass. Recorded as DS-R2.

Results — language identification:

Track	Accuracy
English	100.00%
Sinhala	100.00%
Tamil	100.00%
Singlish	91.56%
Tamilish	25.63%

Tamilish identification is unchanged from the previously recorded measurement and remains open as DS-R1. In the authenticated path this is mitigated because follow-up turns use the stored ticket language rather than re-detecting, but any path that must infer language from text alone is unreliable for Tamilish.

3.1.5 Load Testing

Technique Objective	Observe system behaviour under concurrent authenticated use and establish a throughput and latency profile for the endpoints exercised.
Technique	Drive the running container stack with 20 concurrent authenticated customers, a 5 s ramp and 20 iterations each, issuing five read-only requests per iteration: health, readiness, current user, ticket list and ticket detail. The bearer credential is supplied at runtime and is not persisted in the test plan file.
Oracles	Self-verifying. Per-sample success flag, response code and elapsed time in the result log, recomputed independently for this report.
Required Tools	Apache JMeter, the business load test plan, and its runner script.
Success Criteria	Zero errors across the run and stable latency at the target concurrency.
Special Considerations	This exercises read-only paths only. No write transaction, password hashing, attachment upload, OCR, classification or generation was placed under

	concurrent load.
--	------------------

Results, recomputed from the raw result log — 2,000 samples, all HTTP 200:

Measure	Value
Samples	2,000
Errors	0 (0.00%)
Mean	17.0 ms
Minimum and maximum	1 ms and 101 ms
p50 / p90 / p95 / p99	14 / 30 / 39 / 70 ms
Throughput	approximately 346 requests per second

Endpoint	Samples	Mean	Maximum
Health	400	8.0 ms	88 ms
Readiness	400	13.2 ms	60 ms
Current user	400	13.8 ms	76 ms
Ticket list	400	25.4 ms	101 ms
Ticket detail	400	24.4 ms	100 ms

Conclusion. The authenticated read path is stable at 20 concurrent users with no errors, an improvement on the previous cycle, which loaded only an unauthenticated health endpoint. It supports no capacity claim for the write and generation paths: ticket creation and assistance generation are measured at 2.1 s and 26.6 s for a single user and have not been measured under concurrency. Recorded as LOAD-01.

3.1.6 Security and Access Control Testing

Technique Objective	Verify that each actor type reaches only the functions and data its role permits, that abuse paths are rate-limited, and that prompt-injection and safety routing behave identically across all five language tracks.
Technique	Execute the security suite covering authorisation and roles, token misuse, insecure direct object references, input validation, uploads and rate limiting; re-measure the prompt-injection bypass corpus offline, importing the corpus directly from the test module so that filter and test cannot drift apart; re-measure paired safety coverage across all five renderings of 3,079 paired tickets; re-measure false-positive load over 15,395 real tickets; run passive dynamic scans against the API and the frontend; run static analysis and dependency audits.
Oracles	Self-verifying. Test runner results; the offline diagnostic's detection rate, structural-gap booleans, paired coverage rates and false-positive counts; scanner alert risk and confidence per site.
Required Tools	Security-marked test suite; the offline guardrail diagnostic; OWASP ZAP; Ruff; mypy; JavaScript dependency audit.
Success Criteria	Role and token controls pass; every abuse path is rate-limited; the injection corpus is fully detected; safety routing fires on all five renderings of a

	flagged ticket or on none; no medium or high dynamic-scan finding.
Special Considerations	Dynamic scans are unauthenticated and passive; they complement rather than replace the role-aware suite. Python static analysis and Python dependency auditing were not executed in this cycle.

Results — behavioural controls. The security suite passes: forged, expired and unsigned tokens; refresh-token misuse; inactive users; role restrictions; insecure direct object references; upload type and signature validation; path traversal; null-byte filenames; cross-site-scripting-shaped payloads; SQL injection; cross-origin behaviour; request identifiers; and redaction of passwords, bearer tokens and card numbers from telemetry.

Rate limiting is implemented and covered by eight passing cases: repeated failed logins throttled, account lockout under sustained brute force, registration limited, ticket creation limited, attachment upload limited, assistance endpoint limited, failed login not revealing whether an account exists, and oversized uploads refused. All six previously recorded rate-limit gaps are closed.

Results — prompt injection and multilingual safety. This is the most significant verified improvement in the release:

Measurement	Previously recorded	Measured now
Injection corpus detection	3 of 18 (16.67%)	18 of 18 (100%)
Payload caught as a direct query	Yes	Yes
Payload caught inside ticket context	No	Yes
English tickets escalating on safety	113	841
Sinhala coverage against English	3.54%	100%
Singlish coverage against English	3.54%	100%
Tamil coverage against English	2.65%	99.88%
Tamilish coverage against English	3.54%	99.88%
Flagged in English, missed in both Sinhala and Tamilish	109	0

The guardrail now carries native Sinhala and Tamil injection patterns, and the customer-controlled ticket-context field is passed to the filter. The previously recorded language-equity failure is closed.

The strict success criterion is nonetheless not met. The criterion requires safety routing to fire on all five renderings of a flagged ticket or on none. One ticket of 841 breaks that rule:

Field	Value
Ticket identifier	1410
Category	compromised_card
English escalation reason	private_account_data
Escalates in	English, Sinhala, Singlish
Does not escalate in	Tamil, Tamilish

This is a genuine unauthorized-transaction report that reaches generation in the two Tamil tracks while being escalated to a person in the other three. The "missed in both Sinhala and Tamilish" counter reads zero only because Sinhala catches it; that counter cannot detect a Tamil-only gap. The criterion is recorded as unmet at a measured tolerance of 1 in 841, or 0.12%, and carried as SAFE-02. It does not reopen the original critical finding, whose severity came from 109 tickets missed in both non-English scripts.

Results — false-positive and escalation load, over 15,395 tickets:

Track	Injection false positives	Rate	Safety escalations
English	3	0.097%	115
Sinhala	0	0.000%	5
Tamil	0	0.000%	75
Singlish	0	0.000%	18
Tamilish	0	0.000%	42

The injection filter is tight. The safety router is deliberately broad: under category-aware routing, 841 of 3,079 paired tickets, or 27.3%, escalate to a person. That is intended for account-data and financial-action categories, but it is a staffing consequence that should be sized before deployment.

Results — dynamic scanning.

Scan	Findings
API	4 informational alerts; no medium or high. Client-error responses from unauthenticated probing, authentication request identified, non-storable content, storable and cacheable content.
Frontend	3 medium-risk, high-confidence alerts plus 1 informational: content-security-policy wildcard directive, content-security-policy inline styles permitted, and missing sub-resource integrity.

The three frontend findings are recorded as WEB-02.

Results — static and dependency analysis.

Gate	Result
Ruff	Clean
mypy, strict	Clean across 38 source files
JavaScript dependency audit	0 vulnerabilities at every severity
Python static analysis	Not executed in this cycle
Python dependency audit	Not executed in this cycle

RAG answer quality and retrieval structure were not measured. Injection handling and safety routing are established above. Retrieval accuracy, citation correctness, faithfulness and generation latency are not, and the structural probe covering chunking, citation validation and the confidence gate was not re-run. Its last recorded output identified one citation false accept, one false reject, an unconditional confidence floor for a single approved chunk, and a discontinuity in the confidence score at a dense score of exactly zero. Whether those four were addressed is unknown; they are carried as RAG-R2.

3.1.7 Failover and Recovery Testing

Technique Objective	Simulate failure conditions and exercise recovery processes, to observe whether the system returns to a known good state without manual repair or data loss.
Technique	Restart the API container while the stack is running, then poll health and readiness endpoints until they return a successful status, and confirm that no operator action was required.
Oracles	Self-verifying at a coarse level: a successful health response and a successful readiness response after

	restart, with no manual intervention.
Required Tools	Docker Compose; health and readiness endpoints.
Success Criteria	The service returns to a healthy state automatically after process restart.
Special Considerations	This validates process restart only. It is not failover testing in the sense this plan defines.

Results. The API container was restarted through Compose. Two transient connection resets occurred during restart, after which health and readiness both returned successfully without manual repair. This result is carried forward from the previous cycle and was not re-executed here.

Not exercised in any cycle. Database restore; cache outage behaviour; interrupted uploads; worker replay and idempotency; volume backup and restore; provider failover; multi-instance failover; and the corrupted-pointer and invalid-key scenarios this plan calls for. Recovery capability beyond single-process restart is unknown, and this is the largest untested area of the plan. Recorded as REC-01.

3.1.8 Configuration Testing

Technique Objective	Verify that the system operates across the supported software configurations, and identify configuration state that changes behaviour between developer environment, container image and continuous integration.
Technique	Validate the Compose configuration and service resolution; build and start the stack; compare the local toolchain against the container runtime; execute the backend suite on a second operating system and Python minor version to surface host-dependent behaviour.
Oracles	Self-verifying. Configuration validation exit status and resolved service list; container health checks; test results on each platform.
Required Tools	Docker Compose; the project environment; the test runner on both platforms.
Success Criteria	The configuration resolves and the stack starts healthy; the suite produces equivalent results across supported platforms.
Special Considerations	Cross-platform execution is the purpose of this technique. Differences found here are defects, not environmental noise.

Results — configuration. Configuration validation passes and resolves five services: database, cache, API, frontend and worker. The stack builds and starts healthy on Python 3.12. The runtime image and the type-checking target now agree at Python 3.12, closing the previously recorded version mismatch; the project metadata still admits Python 3.11, so continuous integration must pin 3.12 for that agreement to hold.

Results — cross-platform execution. Running the suite on a second platform with Python 3.13 surfaced two defects that do not appear on the primary platform:

- The raw-SQL interpolation scan in the database integrity suite.
- The evaluation case-file field check.

Both fail with a character-decoding error. The cause is a file read without an explicit encoding, which resolves to the platform default and fails on any non-ASCII byte in a scanned source or case file. The fix is to pass an explicit UTF-8 encoding at each call site.

The practical significance exceeds the severity: the security-relevant raw-SQL scan does not run at all on the affected platform, so it silently provides no protection there. Recorded as CFG-01.

Results — runner portability. The frontend suite requires a single-threaded pool to complete on the second platform; the default parallel pool times out workers and reports only part of the suite. Recorded as CFG-02.

4. Deliverables

4.1 Test Evaluation Summaries

This document is the master evaluation summary for the cycle, produced once per test cycle in Word format generated from the supplied template, with the Markdown source retained alongside it for review.

Each summary contains an executive table with one row per test area and an explicit basis for each result; the technique sections with their results; requirement traceability; a defect register with severity and status; and residual risk with an acceptance decision.

The evidence artifacts delivered with this cycle:

Area	Artifact
API business flow	newman-final.json, swift_complete.postman_collection.json
Load	jmeter-business-final.jtl, business_load_test.jmx, run_business_load.py, jmeter-business-final-report/
Browser	cypress-final.xml, cypress/e2e/login.cy.js, cypress.config.js, selenium_smoke.py, selenium_mobile.png
Accessibility and page quality	axe-final.json, lighthouse-final.json
Dynamic security	zap-api-final.json, zap-frontend-final.json, and their HTML reports, zap.yaml
Dataset	ds-validation-final.json, data-cleaning-final.txt, ds_validation.py
Safety and language diagnostics	offline_guardrails-v3.json, offline_language-v3.json
Backend and frontend execution	pytest-v3.txt, coverage-v3.json, vitest-v3.json, npm-audit-v3.json
Classifier results	ml/reports/RESULTS.md and the saved run records
Report generation	format_test_report.py, build_v3_docx.py

All artifacts are held under `Testing/test_evidence/` except the classifier results, which remain with the modelling reports. Tool output written to the project's report directories is not version-controlled, so any result not listed above is not retained and cannot be audited after the fact. Recorded as EVID-01.

4.2 Reporting on Test Coverage

Coverage is reported at three levels, refreshed once per cycle.

Code coverage. Three distinct metrics are reported, because conflating them is how coverage claims drift. *Statement coverage* is executed statements over total statements. *Branch coverage* is taken branch outcomes over total branch outcomes. *Combined coverage* is covered statements plus covered branches, over total statements plus total branches; it is always the lowest of the three and must not be quoted as line or statement coverage.

Component	Statement	Branch	Combined	Function
Backend	71.72% (1,613 of	43.41% (178 of	67.36% (1,791 of	—

Component	Statement	Branch	Combined	Function
	2,249)	410)	2,659)	
Frontend	19.59%	19.14%	—	15.25%

Frontend line coverage is 21.82%. Backend statement coverage leaves 636 statements uncovered, and 50 of the 410 branches are only partially taken. The weakest modules are the ingest path at 0%, the evaluation runner at 32%, the API route module at 33%, the OCR module at 52% and the judge module at 55%. Backend branch coverage would rise with a live database and generation provider available, which would exercise the evaluation runner and parts of the route module.

No coverage threshold is configured as a failing gate. Recommended incremental minimums are 80% backend statement and 60% backend branch, and 70% frontend line and 50% frontend branch for authenticated flows, introduced gradually so that aggregate numbers are not gamed.

Requirement coverage is reported in §4.3.

Evidence coverage. Every result in this report was either executed for this cycle or read from a retained evidence file, with two exceptions: Python static analysis with dependency auditing, and RAG retrieval with answer quality. Both were not executed and are reported as such wherever they appear.

4.3 Requirement Traceability Matrix

Each use case is mapped to the API surface that implements it, the test module that covers it, the result, and the evidence. Use-case identifiers are assigned in this document; the project does not maintain a separate requirements register, and creating one is recommended in §5.6. Counts are declared test functions, which parametrisation expands to the 199 cases collected.

Use case	API surface	Test module	Cases	Result	Evidence
UC-01 Register a customer account	POST /auth/register	authorisation, input validation	31	Pass	pytest-v3.txt, newman-final.json
UC-02 Authenticate and obtain a token	POST /auth/login, /refresh, /logout	authorisation	16	Pass	pytest-v3.txt
UC-03 Reject anonymous and forged access	all protected routes	authorisation	16	Pass	pytest-v3.txt, newman-final.json
UC-04 Enforce role separation	/admin/*, /tickets, /dashboard/metrics	authorisation, policies	19	Pass	pytest-v3.txt
UC-05 Submit and classify a ticket	POST /tickets	inference, attachment intent	16	Pass	pytest-v3.txt, newman-final.json
UC-06 Retrieve only tickets the caller owns	GET /tickets, /tickets/{id}	authorisation, database integrity	35	Pass	pytest-v3.txt, newman-final.json
UC-07 Upload an attachment and read it by OCR	POST /tickets/{id} /attachments, download	attachment intent, input validation	26	Pass	pytest-v3.txt
UC-08 Mask sensitive spans before inference	POST /ocr/test-masking	attachment intent	11	Pass	pytest-v3.txt

Use case	API surface	Test module	Cases	Result	Evidence
UC-09 Assign, escalate and change ticket status	assignment, status, escalate, notes	policies	3	Pass	pytest-v3.txt
UC-10 Generate a retrieval-grounded draft	assistance endpoint	RAG, RAG API	35	Pass	pytest-v3.txt, newman-final.json
UC-11 Validate citations and fail closed	citation and validation modules	RAG	32	Pass	pytest-v3.txt
UC-12 Block prompt injection in every language	guardrail module	prompt injection	9	Pass, 18 of 18 corpus	pytest-v3.txt, offline_guardrails-v3.json
UC-13 Escalate account and financial requests	safety module	prompt injection	9	Conditional, 840 of 841	offline_guardrails-v3.json
UC-14 Rate-limit and lock out abuse paths	login, register, tickets, uploads, assistance	rate limiting	8	Pass	pytest-v3.txt
UC-15 Keep the published contract honest	GET /openapi.json	contract	9	Pass, 1 skip	pytest-v3.txt, newman-final.json
UC-16 Preserve data integrity and migrations	persistence layer, migrations	database integrity	19	1 fail, CFG-01	pytest-v3.txt
UC-17 Trace requests without leaking secrets	instrumentation	observability	12	Pass	pytest-v3.txt
UC-18 Evaluate retrieval against golden cases	evaluation runner	evaluation runner	4	1 fail, 2 skip	pytest-v3.txt
UC-19 Render and validate the customer interface	React pages and primitives	frontend suite, 18 files	45	Pass	vitest-v3.json, cypress-final.xml
UC-20 Meet automated accessibility rules	production login route	axe-core, Lighthouse	—	Pass	axe-final.json, lighthouse-final.json
UC-21 Serve concurrent authenticated reads	five read-only endpoints	load test plan	2,000 samples	Pass	jmeter-business-final.jtl
UC-22 Serve concurrent writes and generation	ticket creation, OCR, assistance	none	0	Not covered, LOAD-01	—
UC-23 Recover	container stack	none	0	Not covered,	—

Use case	API surface	Test module	Cases	Result	Evidence
from component failure				REC-01	
UC-24 Keep the five language tracks aligned	dataset build pipeline	dataset and cleaning audits	10 files	Pass	ds-validation-final.json, data-cleaning-final.txt

UC-22 and UC-23 have no tests. They are listed rather than omitted so that the untested surface appears in the same table as the tested surface.

5. Risks, Dependencies, Assumptions, and Constraints

5.1 Defect register

ID	Severity	Area	Finding	Status	Recommended action
SAFE-02	Medium	Safety parity	Ticket 1410, an unauthorized-transaction report, escalates in English, Sinhala and Singlish but reaches generation in Tamil and Tamilish; strict parity unmet at 840 of 841	Open	Add ticket 1410 as a named regression case; extend the Tamil-script and Tamil-romanized account-data patterns until parity reaches 841 of 841
WEB-02	Medium	Web hardening	Three medium-risk frontend findings: content-security-policy wildcard, inline styles permitted, and missing sub-resource integrity	Open	Self-host fonts, add sub-resource integrity, remove the wildcard and inline-style allowances, then rescan
CFG-01	Medium	Cross-platform	Files read without an explicit encoding fail on platforms whose default is not UTF-8; two tests fail, including the raw-SQL interpolation scan, which therefore does	Open	Pass an explicit UTF-8 encoding at every call site; add a second-platform job to continuous integration

ID	Severity	Area	Finding	Status	Recommended action
			not run there		
RAG-R1	Medium	RAG quality	Answer quality, citation correctness and generation latency are unmeasured; assistance takes 26.6 s for a single user	Open	Run the golden and development evaluations and retain the output; optimise latency without weakening citation controls
RAG-R2	Medium	RAG structure	The structural probe was not re-run, so four previously recorded citation and confidence-gate defects are neither confirmed fixed nor confirmed open	Open	Re-run the structural probe at the release build and retain the output
LOAD-01	Medium	Performance	Only read-only endpoints load-tested; ticket creation and assistance generation unmeasured under concurrency	Open	Add scenarios for login, ticket creation, upload and OCR, search and assistance
REC-01	Medium	Recovery	Only process restart tested; database restore, cache outage, worker replay, backup and restore, and provider failover untested	Open	Exercise outage, restart, replay, backup and restore, and idempotency
DS-R1	Medium	Language	Tamilish language identification 25.63%; Singlish 91.56%	Open	Carry the stored ticket language end to end; retrain or replace the heuristic detector
DS-R2	Medium	Model quality	Tamilish intent error 29.49% with LaBSE, 37.12% with the	Open	Native-speaker sample audit, spelling normalisation

ID	Severity	Area	Finding	Status	Recommended action
			classical model		and augmentation, per-intent confusion review
TEST-R1	Medium	Frontend quality	Frontend coverage 21.82% line; page and service layers near zero	Open	Add authenticated customer, agent and administrator workflow tests and request-mocking fixtures
EVID-01	Medium	Process	Tool output is written to directories excluded from version control, so results are not retained and cannot be audited later	Open	Write all tool output to the tracked evidence directory
DATA-01	Low	Dataset	6 leakage rows; 1 conflicting-label group of 4 rows; 218 exact-duplicate rows	Open, disclosed	Human review without silent deduplication; sensitivity analysis preserving the frozen split
A11Y-02	Low	Accessibility	One accessibility item requiring manual adjudication; no screen-reader, keyboard-only or zoom and reflow pass	Open	Adjudicate manually; run a screen reader and keyboard traversal
CFG-02	Low	Continuous integration	The frontend runner's default parallel pool times out workers on a second platform	Open	Pin the single-threaded pool or raise the worker timeout
SAST-01	Low	Process	Python static analysis and Python dependency auditing were not executed	Open	Add both to the development extra and to continuous integration, retaining their

ID	Severity	Area	Finding	Status	Recommended action
					output

Defects closed and verified

Basis states how each closure was established: *Executed* — the check was run for this report; *Artifact* — a retained evidence file was read; *Read* — project source or documentation was inspected.

ID	Original severity	Defect	Basis	Verification
SEC-01	Critical	Multilingual prompt-injection bypass, 15 of 18	Executed	18 of 18 detected; native Sinhala and Tamil patterns present in the guardrail
SEC-01b	Critical	Ticket context reached the prompt unscanned	Executed	Payload now caught when placed in ticket context
SAFE-01	Critical	Multilingual safety parity 2.65–3.54%; 109 tickets missed in both Sinhala and Tamilish	Executed	840 of 841 parity; 0 missed in both. Residual single-ticket gap carried as SAFE-02
SEC-02	High	Six abuse paths without rate limits	Executed	Eight rate-limit and lockout cases pass
API-01	High	Published contract drift	Executed	No expected-failure markers remain; contract suite passes
DATA-02	High	Tamilish training row with a divergent category	Artifact	Cross-language label alignment passes on 10 of 10 files
DATA-03	Low	Seven line-ending differences in source text	Artifact	Source-text alignment passes on 10 of 10 files
DS-01	High	Current predictions reported against superseded label tables	Read	Published results headline the current label version and mark earlier rows non-comparable
A11Y-01	Medium	Missing level-one heading; undersized touch targets	Artifact	Zero accessibility violations; accessibility score 100
PERF-01	Medium	Page performance score 76; First Contentful Paint 4.0 s	Artifact	Performance 95; First Contentful Paint 2.2 s
CFG-03	Medium	Python version mismatch between runtime image and type checking	Read	Runtime image and type-checking target both Python 3.12

5.2 Risks, mitigation and contingency

Risk	Likelihood	Impact	Mitigation strategy	Contingency if realised
Capacity inferred from read-only load figures	High	High	State the read-only scope wherever throughput is quoted	Re-scope load testing to write and generation paths before any capacity commitment
Answer quality assumed from safety results	Medium	High	Report safety and quality separately; mark quality unmeasured	Withhold customer-facing generation until the golden evaluation is run and retained
Evidence not retained, so results cannot be audited	High	Medium	Write tool output to the tracked evidence directory	Re-run affected gates; treat unretained results as unverified
Tamilish underperformance reaches production	Medium	High	Subgroup thresholds; native-speaker audit; disclose in demonstrations	Route low-confidence Tamilish tickets directly to a human agent
Broad safety escalation overloads agents	Medium	Medium	Size the 27.3% escalation rate against agent capacity before deployment	Tune category-aware routing on development data; re-verify parity once on held-out data
Security scan silently skipped on one platform	Medium	Medium	Close CFG-01; add a second-platform job	Treat that platform's results as incomplete until the encoding fix lands
Medium web-hardening findings ship unaddressed	Medium	Medium	Track WEB-02 as a release-gate item	Accept explicitly with a documented compensating control, or delay
Dataset duplicates and leakage bias published metrics	Low	Medium	Disclose the leakage and duplicate rows; preserve the frozen split	Sensitivity analysis; regenerate affected results with provenance

5.3 Dependencies

- A container runtime for the stack, load testing, browser testing and dynamic scanning.
- A local generation provider for assistance and evaluation evidence.
- PostgreSQL with pgvector and an ingested knowledge base for the two skipped golden retrieval tests.
- Optional contract-fuzzing, Python static-analysis and Python dependency-audit packages in the development extra; currently absent from the default environment.
- Third-party generation, embedding and vision providers, which were not called in this cycle.

5.4 Assumptions and constraints

- **Assumption:** the five language tracks are renderings of one ticket set keyed by identifier, so a correct rule fires on all five or none. Every parity measurement depends on this invariant, which the dataset audit confirms holds.
- **Assumption:** sentiment and priority metrics measure agreement with the active labelling prompt version, not human ground truth, and are not directly comparable to a hand-annotated benchmark.
- **Constraint:** the backend and frontend suites, static analysis and offline diagnostics were executed on a developer workstation without the container stack or a generation provider. API, browser, load and dynamic-scan results come from the containerised stack and are identified as such.
- **Constraint:** Python static analysis, Python dependency auditing, the RAG structural probe and the RAG evaluations were not executed in this cycle. No claim is made about their outcome.
- **Constraint:** Tamil is machine-translated without the hand-correction pass applied to Sinhala; the romanized tracks are not equivalent in provenance to the native-script tracks.
- **Constraint:** no bank connection exists. The system must not be represented as authorised to move money, and external financial actions must remain disabled in any demonstration.
- **Constraint:** automated accessibility rules cover only part of WCAG. No blind or low-vision user study was performed.

5.5 Acceptance decision

Intended use	Decision
Research or demonstration with human approval and disclosed limitations	Acceptable
Unsupervised customer-facing reply generation	Not acceptable — answer quality is unmeasured
Production banking-support triage	Not acceptable — LOAD-01, REC-01, RAG-R1 and WEB-02 are open
Publication of classifier results	Acceptable at the current label version on the frozen split, with the leakage and duplicate rows disclosed
Publication of retrieval evaluation results	Not acceptable until the evaluations are run and the output retained

5.6 Ordered remediation and retest plan

1. Pass an explicit UTF-8 encoding at every file-read call site; confirm the raw-SQL scan runs on both platforms; add a second-platform job to continuous integration.
2. Write all tool output to the tracked evidence directory, and add the Python static-analysis, dependency-audit and contract-fuzzing packages to the development extra.
3. Run the structural probe to settle the four open citation and confidence-gate questions; run the golden and development retrieval evaluations and retain the output; reduce assistance latency without weakening citation controls.
4. Add ticket 1410 as a named safety regression case and extend the Tamil-script and Tamil-romanized account-data patterns until paired parity reaches 841 of 841.
5. Close the three frontend hardening findings: self-host fonts, add sub-resource integrity, tighten the content-security policy, then rescan.
6. Extend load testing to login, ticket creation, attachment and OCR, search and assistance, with realistic data volume.
7. Add authenticated customer, agent and administrator workflow tests to the frontend and raise coverage on pages and services.
8. Size the 27.3% safety escalation rate against agent capacity; tune on development data and verify parity once on held-out data.
9. Address Tamilish: carry the stored language end to end, run a native-speaker audit, apply spelling normalisation and augmentation, and set subgroup thresholds.

10. Exercise database restore, cache outage, worker replay, backup and restore, and provider failover.
11. Complete the human review of the leakage rows, the conflicting-label group and the exact duplicates, without silent deletion.
12. Establish a requirements register so that traceability identifiers are maintained outside this document.
13. Re-run this plan in continuous integration on the production Python version, on a Linux container host and on a second platform, requiring WEB-02, CFG-01 and EVID-01 closed before release approval.

6. References

1. Rational Unified Process, *Master Test Plan* template — Testing/Template for Test plan.docx.
2. Testing resources framework sheet — Testing/Testing Resources - Sheet1.pdf.
3. Sample test plan report — Testing/Sample test plan report.pdf.
4. Swift project README.md and backend/TESTING.md.
5. ml/reports/RESULTS.md and the saved modelling run records.
6. docs/rag_error_analysis/findings.md and taxonomy.md.
7. Apache JMeter, available at <https://jmeter.apache.org/> (Accessed on 20 September 2026).
8. Cypress, available at <https://www.cypress.io/> (Accessed on 20 September 2026).
9. Selenium WebDriver, available at <https://www.selenium.dev/> (Accessed on 20 September 2026).
10. Postman and Newman, available at <https://www.postman.com/> (Accessed on 20 September 2026).
11. Deque axe-core, available at <https://github.com/dequelabs/axe-core> (Accessed on 20 September 2026).
12. Google Lighthouse, available at <https://developer.chrome.com/docs/lighthouse/> (Accessed on 20 September 2026).
13. OWASP Zed Attack Proxy, available at <https://www.zaproxy.org/> (Accessed on 20 September 2026).
14. pytest, available at <https://docs.pytest.org/> (Accessed on 20 September 2026).
15. Vitest and React Testing Library, available at <https://vitest.dev/> and <https://testing-library.com/> (Accessed on 20 September 2026).
16. Ruff and mypy, available at <https://docs.astral.sh/ruff/> and <https://mypy-lang.org/> (Accessed on 20 September 2026).
17. OWASP API Security Top 10, available at <https://owasp.org/API-Security/> (Accessed on 20 September 2026).
18. Web Content Accessibility Guidelines 2.2, W3C Recommendation, available at <https://www.w3.org/TR/WCAG22/> (Accessed on 20 September 2026).

Appendix A — Commands Executed

Executed against the build under test, with output retained in Testing/test_evidence/.

pytest -q --cov=app --cov-report=json	195 passed, 2 failed, 3 skipped
pytest --collect-only	199 collected
ruff check app tests	clean
mypy app	clean, 38 source files
offline guardrail diagnostic	18 of 18 detection; paired parity 840 of 841
offline language diagnostic	Tamilish 25.63%, Singlish 91.56%
vitest run	45 of 45, 18 files
vitest run --coverage	21.82% line
npm run lint / typecheck / build	all pass
npm audit --json	0 vulnerabilities
docker compose config	valid; five services

Results read and recomputed from retained artifacts rather than re-executed: the API business flow, the load result log, the accessibility and page-quality audits, the browser suite, the dataset and cleaning audits, and the two dynamic scans.